

Base Package Architecture Migration Plan

1. Repository Structure

Base Repository

- Single base repository containing all shared/core logic
- Multiple project modules housed within (e.g., `healthspark`, `molina`, etc.)
- Child projects import via: `from base.healthspark import healthspark`

Directory Layout

```
base/
├── core/           # Shared utilities used by all projects
│   ├── request_handler.py
│   ├── parser.py
│   ├── db_client.py
│   └── utils.py
├── healthspark/
│   ├── __init__.py
│   └── ...
├── molina/
│   ├── __init__.py
│   └── ...
└── pyproject.toml
```

2. Core Logic Centralization

- All shared dependencies live exclusively in the base repository
- Child projects contain only project-specific logic
- No duplication of core utilities across child projects

3. Configuration Management

Each child project maintains its own isolated configuration:

```
child_project/
├── config/
│   ├── .env
│   ├── settings.py
│   └── constants.py
└── ...
```

- Environment variables scoped per project
 - No cross-contamination of configs between projects
-

4. Versioning Strategy

Problem

Direct imports from latest base code can introduce breaking changes.

Solution: Pinned/Cached Dependencies

Option A: Semantic Versioning + PyPI (Recommended)

```
toml

# child project's pyproject.toml
[dependencies]
base = "==1.2.3" # Pin to specific version
```

Option B: Git Submodules

- Lock to specific commit hash
- Requires manual updates
- More complex workflow

Option C: Git Tags + pip install

```
bash

pip install git+https://github.com/org/base.git@v1.2.3
```

Recommendation

- Use semantic versioning (MAJOR.MINOR.PATCH)
 - Publish base package to private PyPI or use tagged releases
 - Child projects pin to specific versions
 - Update deliberately, not automatically
-

5. Code Organization for Readability

Current Approach

```
project/  
└─ index_1.py # 500+ lines, multiple responsibilities
```

Proposed Approach

```
project/  
└─ index_1/  
    ├── __init__.py    # Exports combined functionality  
    ├── api.py         # HTTP request handling only  
    ├── parser.py      # Content parsing only  
    ├── transformer.py  # Data transformation only  
    └─ writer.py       # Output/storage only
```

Benefits

- Single responsibility per file
- Easier debugging and testing
- Better git diffs and code reviews
- Parallel development on different components

`__init__.py` Example

```
python
```

```
from .api import fetch_data
from .parser import parse_response
from .transformer import normalize_records
from .writer import save_to_db
```

```
def run_pipeline(config):
    raw = fetch_data(config)
    parsed = parse_response(raw)
    normalized = normalize_records(parsed)
    return save_to_db(normalized)
```

6. Database Abstraction Layer

Problem

Direct SQLite usage creates tight coupling, making future migrations painful.

Solution: Repository Pattern + Abstraction Layer

```
python

# base/core/db/abstract.py
from abc import ABC, abstractmethod

class BaseRepository(ABC):
    @abstractmethod
    def insert(self, table: str, data: dict) -> int: ...

    @abstractmethod
    def query(self, table: str, filters: dict) -> list: ...

    @abstractmethod
    def bulk_insert(self, table: str, records: list) -> int: ...
```

```
python
```

```
# base/core/db/sqlite.py
class SQLiteRepository(BaseRepository):
    def __init__(self, db_path: str):
        self.conn = sqlite3.connect(db_path)

    def insert(self, table: str, data: dict) -> int:
        # SQLite-specific implementation
        ...
```

```
python

# Future: base/core/db/postgres.py
class PostgresRepository(BaseRepository):
    def __init__(self, connection_string: str):
        self.conn = psycopg2.connect(connection_string)

    def insert(self, table: str, data: dict) -> int:
        # Postgres-specific implementation
        ...
```

Usage in Child Projects

```
python

# Child project just uses the interface
from base.core.db import get_repository

db = get_repository() # Returns SQLite now, Postgres later
db.insert("providers", record)
```

Migration Path

1. Implement abstraction layer with SQLite backend
2. When ready to switch: implement new backend class
3. Change single config value to switch databases
4. Zero changes needed in child projects

7. Implementation Checklist

- ☐ Set up base repository structure
 - ☐ Extract and centralize core utilities
 - ☐ Implement database abstraction layer
 - ☐ Add semantic versioning to base package
 - ☐ Create project template for child repositories
 - ☐ Document import patterns and best practices
 - ☐ Migrate one project as proof of concept
 - ☐ Roll out to remaining projects
-

8. Next Steps

1. **Immediate:** Decide on versioning approach (PyPI vs Git tags)
2. **Week 1:** Set up base repo structure with core utilities
3. **Week 2:** Implement DB abstraction layer
4. **Week 3:** Migrate healthspark as pilot project
5. **Week 4:** Document patterns, create templates for remaining projects